

Defect Metrics Calculator – Proof of Concept (POC)

1. Introduction

Defect metrics are measurements used in software testing and quality assurance to evaluate the quality of an application and the effectiveness of the testing process. These metrics help teams track defects, identify trends, improve testing efficiency, and enhance overall product quality.

The proposed **Defect Metrics Calculator** is a Proof of Concept (POC) designed to automate defect extraction, metric calculation, quality scoring, and reporting using a workflow-based AI orchestration platform such as Langflow integrated with JIRA.

2. Objectives

The key objectives of the POC are:

- Automate defect extraction from JIRA
 - Reduce manual effort in defect analysis
 - Generate defect metrics for releases/sprints
 - Provide quality score and project quality rating
 - Improve release quality visibility
 - Enable structured reporting to management
 - Support year-end productivity and quality analysis
-

3. Common Defect Metrics

3.1 Defect Density

Definition

Defect Density measures the number of defects identified per unit size of the software.

Formula

$$\text{Defect Density} = \frac{\text{Total Defects}}{\text{Size of Software (KLOC, FP, Modules, User Stories)}}$$

Example

- Total Defects = 50
- User Stories = 10

Calculation:

$$\text{Defect Density} = \frac{50}{10} = 5$$

Result:

- 5 defects per user story
-

3.2 Defect Leakage

Definition

Defect Leakage measures the percentage of defects missed during testing but identified later in UAT or Production.

Formula

$$\text{Defect Leakage \%} = \frac{\text{Defects Found After Release}}{\text{Total Defects}} \times 100$$

Interpretation

- Lower defect leakage indicates better testing quality.
 - Higher leakage indicates insufficient testing coverage or process gaps.
-

3.3 Test Effectiveness

Definition

Test Effectiveness measures how efficiently the testing process identifies defects before release.

Formula

$$\text{Test Effectiveness \%} = \frac{\text{Defects Found During Testing}}{\text{Total Defects Found}} \times 100$$

Interpretation

- Higher effectiveness indicates stronger QA validation.
 - Helps measure QA process maturity.
-

4. Current Challenges

Currently, organizations face several operational challenges while preparing defect metrics:

- Manual extraction of defects from JIRA
- Segregation of defects by:
 - Severity
 - Priority
 - Environment
 - Sprint/Release
- Time-consuming metric calculations
- Manual reporting to management
- Inconsistent quality analysis
- Delays in release reporting
- Difficulty in analyzing yearly productivity trends

Defect Classification Parameters

Severity Levels

- Critical
- High
- Medium
- Low

Priority Levels

- Immediate Fix
- High

- Medium
- Low

Environments

- QA
 - UAT
 - Production (PROD)
-

5. POC Vision – Defect Metrics Calculator

The POC aims to provide a centralized, intelligent, and user-friendly module that:

- Extracts defects automatically from JIRA
 - Filters defects based on:
 - Sprint
 - Release
 - Environment
 - Severity
 - Priority
 - Calculates defect metrics automatically
 - Generates:
 - Quality score
 - Quality rating
 - Summary insights
 - Sends structured reports to management
 - Reduces manual effort significantly
-

6. Proposed Solution Architecture

High-Level Workflow

User Input



Langflow UI



JIRA Integration



Defect Extraction



Data Processing & Classification



Metric Calculation Engine



Quality Score Calculation



Report Generation



Email Notification to Stakeholders

7. Langflow Model Approach

The proposed implementation using Langflow includes the following workflow steps:

Step 1 – Extract Defects from JIRA

- Connect to JIRA APIs
- Fetch defects for selected sprint/release
- Retrieve metadata:
 - Severity
 - Priority
 - Status
 - Environment
 - Assignee

- Resolution

Step 2 – Process and Categorize Defects

Classify defects based on:

- Severity
- Priority
- Environment
- Sprint
- Release

Example categories:

Category	Example
Severity	Critical, High
Priority	Immediate Fix
Environment	QA, UAT, PROD

Step 3 – Calculate Metrics

Metrics calculated:

- Defect Density
 - Defect Leakage
 - Test Effectiveness
 - Severity Distribution
 - Priority Distribution
 - Environment-wise Defects
-

Step 4 – Generate Quality Score

A weighted scoring mechanism can be used.

Example Criteria

Metric	Weight
Defect Leakage	40%
Critical Defects	30%
Test Effectiveness	20%
Reopened Defects	10%

Step 5 – Assign Quality Rating

Example Rating Logic

Score Range	Rating
85 – 100	Good
60 – 84	Medium
Below 60	Low

Step 6 – Generate Structured Report

The system generates:

- Summary report
- Metrics dashboard
- Trend analysis
- Quality insights
- Management-ready output

Possible formats:

- HTML
- PDF
- Excel
- Email summary

Step 7 – Email Notification

Automatically trigger notifications to:

- Business stakeholders
- QA managers
- Product owners
- Release managers

The email can contain:

- Sprint summary
- Quality score
- Defect Metrics
- Release recommendation

8. Proposed User Interface

Input Parameters

The UI should capture:

Field	Description
Project	JIRA Project
Sprint/Release	Sprint selection
Environment	QA/UAT/PROD
Severity Filter	Optional

Field	Description
Priority Filter	Optional
Date Range	Optional

Output Dashboard

The dashboard can display:

- Total defects
- Severity breakdown
- Priority breakdown
- Defect leakage
- Test effectiveness
- Quality score
- Project quality rating
- Graphs and trends

9. Sample Output

Sprint Quality Summary

Metric	Value
Total Defects	120
Critical Defects	5
Defect Density	4.5
Defect Leakage	8%
Test Effectiveness	92%
Quality Score	88

Metric	Value
Quality Rating	Good

10. Benefits of the Solution

Business Benefits

- Faster reporting
- Better release decisions
- Improved transparency
- Reduced production defects
- Improved stakeholder communication

QA Benefits

- Reduced manual effort
- Better productivity tracking
- Improved defect analysis
- Better sprint quality monitoring

Organizational Benefits

- Improved process maturity
 - Historical quality tracking
 - Data-driven decision making
 - Enhanced software quality
-

11. Future Enhancements

Potential future enhancements include:

- AI-based defect prediction
- Trend analytics dashboard

- Root cause analysis
 - Automated release recommendation
 - Integration with CI/CD pipelines
 - Predictive quality scoring
 - Real-time dashboards
 - Multi-project comparison
-

12. Technology Stack

Component	Technology
Workflow Engine	Langflow
Issue Tracking	JIRA
Backend APIs	Python / FastAPI
Database	PostgreSQL / MongoDB
Visualization	HTML / Charts
Notifications	SMTP / Outlook APIs
Reporting	PDF / Excel / HTML

13. Conclusion

The **Defect Metrics Calculator POC** provides an intelligent and automated approach for defect analysis, quality measurement, and release reporting. By integrating Langflow with JIRA, organizations can significantly reduce manual effort, improve software quality visibility, and provide management with actionable insights for better release governance.